

ASSIGNMENT 1

Real-Time Collaborative Kanban with Autonomous AI Project Manager

(Web App + Chrome Extension)

Overview

You will build a fully real-time collaborative project management web application where multiple users can work on the same board simultaneously with no refresh required. On top of the real-time collaboration layer, an AI project manager runs as a background process that analyses the board state on a schedule, detects bottlenecks and sprint risks, infers task complexity from descriptions, auto-assigns tasks based on team member history, and generates a structured weekly digest report. A GitHub Issues scraper can import tasks directly from any public repository. A Chrome extension lets users clip content from any webpage into a task without leaving their browser.

The UI must be polished and intuitive - this is a product a real team would pay for, and it must look and feel like one.

Section 1: Real-Time Collaboration Engine

Multiple users must be able to work on the same board simultaneously. When user A drags a card from In Progress to Done, user B must see it move in real time without refreshing. When user A edits a card title, user B must see the update appear as it is typed.

Concurrent edits to the same card must be handled without data loss. If user A and user B both edit the same card title simultaneously, the system must not silently discard one edit or produce a corrupt merged result. You must implement a last-write-wins strategy with a visible conflict notification, or an operational transformation approach. The chosen approach must be documented in the README.

The collaboration must work for at minimum 10 simultaneous users on one board without performance degradation. You must test this and document the result.

Board state must persist across server restarts. A user who has items on a board must see those items after the Railway service redeploys.

Section 2: AI Project Manager

The AI project manager runs as a background process on a configurable schedule (default every 6 hours) and performs the following analyses on each board.

Bottleneck detection: identify columns where cards are accumulating faster than they are leaving. A column where 5 new cards arrived this week and only 1 left is a bottleneck. The

AI must not just flag the bottleneck but also identify the likely cause - is it one assignee who is overloaded, a particular label that keeps getting stuck, or a dependency on another column?

Sprint risk assessment: if the board has a sprint end date configured, the AI must assess whether the current velocity (cards completed per day) is sufficient to complete all cards before the deadline, and surface a plain-English risk summary.

Task complexity inference: when a new card is created with a description, the AI must infer its complexity on a configurable scale (for example 1 to 5 story points) based on the description text, the label, and how similar past cards were resolved. The inferred complexity must be displayed as a suggestion that the user can accept or override.

Auto-assignment: when a card has no assignee, the AI must suggest an assignee based on that team member's past completion patterns - who has historically completed similar tasks fastest, who has the lowest current in-progress count, and who has relevant label history. The suggestion must be a suggestion, not an automatic assignment without user confirmation.

All AI analyses must stream their results to a dedicated AI insights panel in the UI as they complete, not appear all at once.

Section 3: GitHub Issues Scraper

The system must allow a user to paste any public GitHub repository URL and import its open issues as cards. The scraper must fetch all open issues including their title, body, labels, assignees, and milestone. It must paginate correctly for repositories with more than 30 issues. Labels must be mapped to the board's label system. Assignees must be mapped to board members by GitHub username if a matching board member exists, or left unassigned otherwise. The import must be incremental: running it twice on the same repository must not create duplicate cards.

Section 4: Chrome Extension

The Chrome extension must allow a user to clip content from any webpage into a new task on their board. When the user selects text on any page and clicks the extension icon, a popup appears with the selected text pre-filled into the task description field, the source URL pre-filled into a reference link field, and a board and column selector. The user can edit the title and description, select a board and column, and click create. The task must appear on the board in real time without the user opening the web app.

The extension must also support clipping an entire page - if the user clicks the extension icon without selecting text, the extension must extract the page title and URL and pre-fill them into the task form.

Because this assignment includes the Chrome extension component, the auto-assignment AI feature is optional rather than required. All other AI project manager features remain required.

Section 5: UI Requirements

A board view with drag-and-drop columns, card creation inline without a modal for speed, card detail modal showing description, assignee, labels, complexity score, comment thread, and activity history. Dragging a card between columns must update all other connected users in real time.

A team view showing each team member's current task load, completion rate this sprint, and label specialisation inferred from history.

An AI insights panel (sidebar or dedicated tab) showing the latest bottleneck analysis, sprint risk summary, and any pending auto-assignment suggestions waiting for approval.

A digest report view showing the last generated weekly report with completion metrics, velocity trend, and top bottleneck identified.

A GitHub import panel where the user pastes a repository URL, previews the issues to be imported with a count and sample titles, and confirms the import.

All views must be polished, responsive, and require no explanation for a developer who has used any project management tool before.

Constraints

All services on Railway free tier. Real-time collaboration must use WebSockets, not polling. All AI inference CPU-bound within Railway memory limits. Chrome extension loadable in developer mode in one step. Live Railway URL at submission time. Public GitHub repository.

Day-by-Day Timeline

Day 1: Database schema designed, board and card CRUD working, WebSocket server running and broadcasting basic events. Two browser windows open on the same board show the same cards. By end of day basic real-time sync works.

Day 2: Drag and drop working with real-time sync across clients. Concurrent edit conflict handling implemented and tested. Board state persists across server restarts. GitHub Issues scraper working with pagination and deduplication.

Day 3: AI project manager background process running on schedule. Bottleneck detection and sprint risk assessment producing meaningful output. Task complexity inference working on new card creation. AI insights streaming to the UI.

Day 4: Chrome extension built and creating tasks in real time from any webpage. Extension tested on at least 10 different websites including pages with paywalls and SPAs. Digest report generation working.

Day 5: Full UI polish across all views. Error handling complete. Concurrent user testing documented. README written. Railway deployment stable.

Bonus Tasks

Board templates: allow users to create a board from a predefined template (software sprint, content calendar, product roadmap) with pre-built columns, label sets, and a sample card structure.

Dependency mapping: allow cards to be linked as blockers of other cards. The AI bottleneck detector must account for dependency chains and flag cards that are blocking multiple downstream tasks.

Time tracking: add a timer to each card that team members can start and stop. The AI project manager must use logged time alongside completion patterns for its complexity inference and auto-assignment suggestions.

Public board sharing: allow a board to be made read-only public with a shareable URL, where anyone can view the board state in real time without an account.

Evaluation Criteria

Whether real-time sync works correctly under concurrent edits from multiple users without data loss. Whether the conflict handling strategy is correctly implemented and documented. Whether the AI bottleneck and sprint risk analyses produce actionable and accurate output on test boards. Whether the GitHub scraper handles pagination and deduplication correctly on large repositories. Whether the Chrome extension works reliably across different webpage structures. Whether the UI is polished enough that a developer would consider using it over existing tools. Whether the board state survives a Railway service restart correctly.

Deliverables

Public GitHub repository. Live Railway URL. Chrome extension zip. README with real-time architecture description, conflict handling approach, AI analysis schedule and methodology, GitHub scraper pagination handling, and concurrent user test results. Screen

recording or written walkthrough (max 4 minutes) showing two simultaneous users editing a board, a GitHub import, the AI insights panel, and the Chrome extension creating a task.